Graph-Based Multi-Objective Routing Algorithm for Carbon-Aware Navigation Systems

SDG: Climate Action

Maps to: DMS, CN, DAA

▼ Basic explanation

What the Topic Means

Graph-Based Multi-Objective Routing Algorithm for Carbon-Aware Navigation Systems

In simple words:

> Design a navigation system (like Google Maps) that chooses routes **not just based on shortest distance or fastest time**, but based on **minimum carbon emissions**.

Most navigation systems optimize:

- **distance**
- **travel time**

But they **ignore environmental impact**.

Your system would compute routes that minimize:

- CO_2 emissions
- fuel consumption
- congestion impact

Example

Suppose you want to go from **A → B**.

Three routes exist.

Route	Distance	Traffic	Emissions
Route 1	5 km	heavy	HIGH
Route 2	7 km	light	LOW
Route 3	6 km	moderate	MEDIUM

Traditional navigation chooses:

👉 **Route 1 (shortest distance)**

But your algorithm might choose:

👉 **Route 2 (least emissions)**

Even though it is longer.

Why Graph Theory is Used

Road networks are naturally modeled as **graphs**.

- **intersections → nodes**
- **roads → edges**

Each road edge has weights like:

- distance
- traffic time
- carbon emission cost

Example graph:

```
A ---- B ---- C
 \     |
  \    |
   D---E
```

Each edge has multiple weights.

The Key Research Challenge

Typical routing algorithms optimize **one value**:

Example:

Shortest path:

```
minimize distance
```

But here you must optimize **multiple objectives simultaneously**.

Example:

```
minimize (distance, emissions, time)
```

This is called **multi-objective optimization**.

The Mathematical Idea

You compute **Pareto optimal routes**.

A route is Pareto optimal if:

> You cannot improve one metric without making another worse.

Example:

Route A

- shorter distance
- higher emissions

Route B

- longer distance
- lower emissions

Both can be optimal depending on priorities.

Relevance to Sustainable Development Goals

This project connects directly to:

SDG 13 — Climate Action

Transportation produces about **24% of global CO_2 emissions**.

Navigation systems could reduce emissions by:

- avoiding stop-and-go traffic
- avoiding congestion
- choosing smoother routes

Even **5–10% emission reduction** across millions of drivers is huge.

Where the Research Comes In

Your research paper could propose something like:

New Carbon-Aware Routing Algorithm

Example ideas:

1. **Weighted multi-objective Dijkstra**
2. **Pareto frontier route search**
3. **Hybrid heuristic combining**
    - travel time
    - emissions
    - fuel consumption

You then compare it with:

- standard Dijkstra
- fastest route algorithms

Metrics:

- total emissions
- travel time
- computational cost

Why This Topic Is Easier to Execute

Compared to other SDG ideas, this one:

1️⃣ Uses public datasets

You can use:

- **OpenStreetMap**
- traffic datasets
- emission models

No hardware needed.

2️⃣ Algorithm-focused

Perfect for **DAA + DMS coursework**.

You implement and analyze algorithms.

3️⃣ Easy evaluation

You can run simulations:

Example:

```
simulate 10,000 trips in a city
compare emissions between algorithms
```

4️⃣ Strong research framing

Paper title could be something like:

"A Multi-Objective Graph Routing Algorithm for Carbon-Aware Urban Navigation Systems."

That sounds like a **real research paper**.

How It Maps to Your Courses

Discrete Mathematical Structures

- Graph theory
- weighted graphs
- Pareto optimality
- combinatorics of routes

Design and Analysis of Algorithms

- Dijkstra algorithm
- multi-objective optimization
- heuristic search
- algorithm complexity analysis

Computer Networks

- routing algorithms
- distributed navigation systems
- network optimization

Real-World Impact

If navigation systems adopt this idea:

Cities could reduce:

- fuel usage
- congestion
- carbon emissions

This makes it a **very strong SDG project**.

Why Professors Like This Type of Project

Because it combines:

- **math (graph theory)**
- **algorithms**
- **real-world impact**
- **simulation experiments**

That is exactly the structure of many **IEEE / ACM student research papers**.

▼ Can include AI?

1. The Core Project (Without AI)

The basic system works like this:

1️⃣ Model the **road network as a graph**

- intersections → nodes
- roads → edges

2️⃣ Each road has weights like:

- distance
- travel time
- carbon emission

3️⃣ Run a **routing algorithm** to find the best route.

Example algorithms:

- Dijkstra
- A*
- Multi-objective shortest path

So the system solves something like:

```
Find route that minimizes:
distance + emissions + congestion
```

This is **pure algorithms + graph theory**, no AI needed.

---

2. Where AI Can Be Added

AI becomes useful when **predicting things that are uncertain**.

AI for Traffic Prediction

Traffic changes constantly.

AI models can predict:

- congestion levels
- travel speed

Example models:

- Random Forest
- LSTM (time series)

Then the routing algorithm uses **predicted traffic**.

AI for Emission Estimation

Fuel consumption depends on:

- speed
- acceleration
- road slope
- traffic stops

AI can estimate **emissions more accurately**.

AI for Adaptive Routing

The system learns which routes reduce emissions over time.

Example:

- Reinforcement Learning

---

3. Two Possible Versions of the Project

Version 1 — Pure Algorithms (Simpler)

Components:

- Graph model of roads
- Multi-objective shortest path algorithm
- Emission cost function

Courses mapped:

- **DMS:** Graph theory
- **DAA:** shortest path algorithms
- **CN:** routing systems

This is already a **valid research paper**.

Version 2 — AI + Algorithms (Stronger Paper)

Components:

1️⃣ AI predicts traffic

2️⃣ AI estimates emissions

3️⃣ Graph algorithm computes best route

Courses mapped:

- **DMS:** graph models
- **DAA:** routing optimization
- **CN:** distributed navigation networks
- **AI/ML:** prediction models

---

4. Example System Architecture

```
Traffic Data  →  AI Model (traffic prediction)
                    ↓
Road Network Graph
                    ↓
Multi-objective Routing Algorithm
                    ↓
Carbon-aware route recommendation
```

---

5. Why This Still Counts as a CS Research Project

Even without AI, the research contribution can be:

- new routing algorithm
- improved heuristic
- multi-objective optimization method
- faster route computation

Many **algorithm research papers do not use AI**.

---

6. What Makes It Publishable

A paper might look like this:

Title

> A Multi-Objective Graph Routing Algorithm for Carbon-Aware Urban Navigation

Contribution

- new routing algorithm
- carbon emission cost model
- comparison with standard routing

Evaluation

- simulation using real road data

Key Insight

In CS research:

- **AI ≠ required**
- **algorithms + graphs + optimization** are often enough.

In fact, **many top algorithm papers contain zero AI**.

▼ Novelty

Your **innovation** can come from:

- a new algorithm
- a better optimization method
- combining two techniques
- applying an algorithm to a **new domain**
- improving performance (speed, emissions, scalability)

Why This Topic Can Still Be Innovative

Yes, **routing algorithms already exist**.

Examples:

- Dijkstra (1959)
- A∗ (1968)
- modern navigation systems

But most routing algorithms optimize **only one objective**.

Typical navigation solves:

$$\min_{path\ P} \sum_{e \in P} w_e$$

Where

w_e = distance or time weight of each road.

So the algorithm finds **the shortest path**.

What Makes Your Version Different

Your system instead solves a **multi-objective optimization problem**.

For example:

$$\min_{P} \big(\alpha \cdot distance + \beta \cdot emissions + \gamma \cdot congestion\big)$$

Where:

- (α, β, γ) = priorities
- emissions depend on **speed, stops, traffic**

So the routing problem becomes **much harder**.

That's where **research contribution** can happen.

Where the Innovation Can Be

You don't invent the entire topic — you innovate **inside it**.

Here are **realistic research contributions**.

---

1. New Carbon Cost Model

Most routing systems use **simple emission estimates**.

You could create a **better carbon model** that includes:

- acceleration
- stop frequency
- road slope
- traffic density

Then integrate that into routing.

Contribution:

> A new **carbon-aware edge weighting model**.

---

2. New Multi-Objective Routing Algorithm

Most algorithms optimize **one metric**.

Multi-objective routing is harder because there may be **many optimal paths**.

Your contribution could be:

- improved Pareto route search
- faster multi-objective Dijkstra
- heuristic pruning

Contribution:

> **Efficient multi-objective pathfinding algorithm**.

---

3. Carbon-Aware Navigation vs Traditional Navigation

You could experimentally show:

- how much CO_2 normal navigation produces
- how much your algorithm reduces it

Example experiment:

Simulate **10,000 trips in a city**.

Compare:

Method	Emissions
Shortest route	high
Fastest route	medium

Method	Emissions
Carbon-aware route	lower

Contribution:

> Evidence that carbon-aware routing reduces emissions.

---

4. City-Scale Simulation

Most research uses **small networks**.

You could test on:

- **OpenStreetMap city graphs**
- thousands of nodes

Contribution:

> scalable algorithm for large road networks.

---

5. Adaptive Emission Routing

Instead of fixed weights, your system could adjust weights dynamically:

Example:

- heavy traffic → increase emission penalty
- smooth roads → reduce penalty

Contribution:

> **adaptive emission-aware routing model**.

What the Research Paper Would Look Like

Typical structure:

1. Problem Statement

Existing navigation systems optimize **distance or time**, not emissions.

---

2. Proposed Method

New **multi-objective routing algorithm**.

---

3. Mathematical Model

Graph-based road network.

---

4. Algorithm Design

Modified shortest-path algorithm.

---

5. Experiments

Simulation on real road data.

---

6. Results

Compare emissions and route efficiency.

Why Professors Accept These Papers

Most CS papers are like:

> "We propose an improved algorithm for X problem."

Not:

> "We discovered a completely new field."

Even famous papers are incremental.

Examples:

- improved routing algorithms
- faster sorting algorithms
- better graph heuristics

Example of a Realistic Paper Title

Something like:

"A Multi-Objective Carbon-Aware Routing Algorithm for Sustainable Urban Navigation."

or

"Improving Urban Route Planning with Emission-Optimized Graph Algorithms."

Key Insight Most Students Miss

Innovation in CS usually means:

- **improving an algorithm**
- **combining known techniques**
- **evaluating them in a new context**

Not inventing something completely new.

▼ Basic implementation strategy

Part 1 — Feasibility of the Carbon-Aware Routing Project

Overall difficulty

For a **3rd semester CSE/ISE student**, this project is **moderately challenging but very feasible** because:

- no hardware needed
- datasets are public
- algorithms already exist as baselines
- you mainly implement and improve them

Most work will be:

- **graph algorithms**
- **data processing**
- **simulation experiments**

Core Idea Recap

The system finds routes that minimize **carbon emissions**, not just distance.

The optimization objective can be written as:

\min_{P} \left(\alpha \cdot distance + \beta \cdot emissions + \gamma \cdot travel\ time\right)

Where:

- (P) = route
- (\alpha, \beta, \gamma) = weights

Modular Architecture of the Project

Think of the project as **5 modules**.

Module 1 — Road Network Graph Builder

What it does

Convert map data into a **graph structure**.

Example:

- intersections → nodes
- roads → edges

Example graph:

```
A ---- B ---- C
 \           /
  \         /
    D ---- E
```

Each edge stores:

- distance
- speed limit
- estimated emissions

Tools

You can use:

- **OpenStreetMap data**
- Python library **OSMnx**
- NetworkX

Output

```
Graph G(V, E)

V = intersections
E = roads
```

Module 2 — Carbon Emission Model

Each road needs a **carbon cost value**.

Basic formula could be:

[

emissions = fuel_rate \times distance

]

But better models include:

- speed

- traffic stops
- vehicle efficiency

Example edge data:

Road	Distance	Speed	Emissions
A→B	1 km	40 km/h	120 g CO_2
B→C	2 km	30 km/h	260 g CO_2

Research opportunity

You could design a **better emission weighting model**.

Module 3 — Routing Algorithm Engine

This is the **core algorithmic component**.

Baseline algorithms:

- **Dijkstra**
- **A**
- **multi-objective shortest path**

Your algorithm will compute routes minimizing:

- emissions
- time
- distance

Example pseudocode idea:

```
for each node:
    compute cost = α * distance + β * emissions
run shortest path algorithm
```

Research opportunity

Design a **new heuristic** or **optimization strategy**.

Module 4 — Simulation Engine

You simulate many trips across the network.

Example experiment:

```
simulate 10,000 trips in Bangalore
```

Compare algorithms:

Algorithm	Avg emissions	Time
Shortest path	high	low
Fastest path	medium	medium
Carbon-aware	lowest	slightly higher

This becomes your **evaluation section in the paper**.

Module 5 — Visualization System

Optional but impressive.

Show routes on a map.

Tools:

- **Leaflet**
- **Python Folium**
- **Mapbox**

Example output:

- shortest route
- green route (low carbon)

Full System Architecture

```
Road Network Dataset
        ↓
Graph Builder
        ↓
Carbon Emission Model
        ↓
Routing Algorithm Engine
        ↓
Simulation Engine
        ↓
Evaluation + Visualization
```

Estimated Workload

If done properly:

Task	Difficulty
Graph building	easy
Emission modeling	medium
Algorithm implementation	medium
Simulation experiments	medium
Visualization	optional

Total time:

4–6 weeks of work.

Why This Project Works for Research

You can contribute in:

- algorithm design
- emission modeling
- scalability experiments

Even small improvements are publishable.

▼ Research Papers

Modeling_and_Analysis_of_CO2_Emissions_of_Vehicles_through_Numerical_Simulation.pdf

Intelligent_Dynamic_Routing_and_Optimization_with_Carbon_Awareness_for_Sustainable_and_Efficient_Network_Management.pdf

Multi-objective_multimodal_transport_path_optimization_model_and_algorithm_considering_carbon_emissions.pdf

Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Considering_Unforeseeable_Road_Eventualities.pdf

Sustainable_and_Optimized_Route_Selection_using_Machine_Learning_Algorithm.pdf

A_carbon-aware_routing_protocol_for_optimizing_carbon_emissions_in_modular_construction_logistics.pdf

Carbon-Aware_Adaptive_Routing_Protocol_with_Real-Time_Traffic_Information_for_Cyber-Physical_Internet_Networks.pdf

Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_Paths.pdf

A_Carbon_Emission_Reduction_Enhancement_Framework_for_Tourism_Routes_Based_on_Time_Window_Optimization.pdf

▼ Github links

https://github.com/search?q=Carbon+aware+routing&type=repositories&p=1

https://github.com/search?q=Multi+objective+shortest+path+&type=repositories

▼ Perplexity analysis

1. Feasibility, difficulty, scope

- **Feasibility:** Very feasible if you:
  - work on a *simulated* city network (OSM / toy network),
  - support 2–3 objectives (distance, time, emissions),
  - target offline routing (no real-time traffic in v1).Carbon-Aware_Adaptive_Routing_Protocol_with_Real-Time_Traffic_Information_for_Cyber-Physical_Int.pdf+2
- **Difficulty level:** Upper-undergraduate / early master's: algorithms + implementation, but no heavy ML needed.Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1
- **Estimated effort (2–3 months, 2–3 students):**
  - Month 1: literature, data + graph pipeline, baseline Dijkstra, simple emissions model.

- Month 2: implement 1–2 multi-objective algorithms, experiment scripts, evaluation.
- Month 3: polish API/UI, run larger experiments, write report/paper.Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+2

- **Recommended scope (for 2–3 months):**
  - One city graph (e.g., Bengaluru or a standard benchmark) with 1000–10,000 nodes.[ppl-ai-file-upload.s3.amazonaws]
  - 3–4 algorithms in total (2 baselines, 1–2 "advanced") and clear evaluation on a fixed OD set.Carbon-Aware_Adaptive_Routing_Protocol_with_Real-Time_Traffic_Information_for_Cyber-Physical_Int.pdf+1

---

2. How to make it novel vs attached papers

From the attached works, we can see:

- Paper focuses on a *two-layer CPI logistics* framework with real-time traffic and a **weighted multi-objective shortest path** (time vs emissions).[ppl-ai-file-upload.s3.amazonaws]
- Paper uses a **hybrid PSO–GA metaheuristic** on real OSM city graphs, but essentially optimizes a *single* scalar "carbon cost" (distance × emission factor).[ppl-ai-file-upload.s3.amazonaws]
- Paper offers a very sophisticated **multimodal multi-objective multi-point shortest path** with evolutionary MMOEA-CDP, focusing on enumerating many Pareto-equivalent paths, not emissions specifically.[ppl-ai-file-upload.s3.amazonaws]
- Paper is an applied **ANN-based eco-journey tracker** focusing on ML-driven emission prediction and dashboarding.[ppl-ai-file-upload.s3.amazonaws]
- Paper is more of an integrated eco-routing *system* with multi-factor heuristic, but primarily at the application level rather than core algorithms.[ppl-ai-file-upload.s3.amazonaws]

So to be novel, you can combine gaps they leave:

Potential novelty directions

1. **Graph-native multi-objective routing with explicit Pareto sets for *navigation use***
   - Instead of just a single weighted sum route, compute and *present* a small Pareto set (e.g., 3–5 non-dominated routes) balancing distance, time, and emissions.Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1
   - Use lightweight multi-objective label-setting / label-correcting on graphs (e.g., MO-Dijkstra / NAMOA-like), not full-blown MMOEA, tuned for navigation scale.[ppl-ai-file-upload.s3.amazonaws]
2. **Scenario-aware / user-aware weights**
   - Integrate a simple *preference model* that maps user preferences (eco vs fast) into weights and show how different weights move along the Pareto front.Carbon-Aware_Adaptive_Routing_Protocol_with_Real-Time_Traffic_Information_for_Cyber-Physical_Int.pdf+1
   - Many attached works fix a weight or optimize only carbon; you can make this interactive.
3. **Transparent emissions model integrated into routing**
   - Implement and expose a modular emissions model like in: base factor × distance × speed-efficiency multiplier (U-shaped vs speed), parameterized by vehicle type.[ppl-ai-file-upload.s3.amazonaws]
   - Most urban eco-routing papers use a linear length×factor model; you can show how speed-dependent factors change routes.Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1
4. **Comparison of exact vs metaheuristic routing for multi-objective eco-routing**
   - Implement both:
     - an *exact* multi-objective label-setting algorithm, and
     - a *metaheuristic* (e.g., simple GA or PSO-GA variant on path encoding).Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1

- Compare on: quality, runtime, scalability, and reproducibility on the *same* road graphs.

5. **Robust / fallback paths for disruptions**
   - Inspired by the multimodal MMMSPP work that preserves multiple equivalent paths for *unforeseeable road eventualities*.[ppl-ai-file-upload.s3.amazonaws]
   - In your system, explicitly output 2–3 robust alternatives and quantify how often they help when you randomly "block" edges.

Any *one* of these, implemented carefully with experiments, is enough for a solid undergrad research paper.

---

3. Modular system architecture

Aim for a clear, pluggable architecture:

1. **Data layer**
   - Road graph loader (e.g., from OSM or a benchmark file): builds directed graph with attributes.Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1
   - Edge attributes: distance (m), free-flow time (s), speed limit, road type, maybe slope if available.[ppl-ai-file-upload.s3.amazonaws]
2. **Emissions model module**
   - Functions:
     - `emission_factor(vehicle_type, speed)` → g CO_2/km.Modeling_and_Analysis_of_CO2_Emissions_of_Vehicles_through_Numerical_Simulation.pdf+1
     - `edge_emission(edge, vehicle_type, traffic_state)` → g CO_2.
   - Start with simple linear models, then optionally add speed-dependent multiplier.[ppl-ai-file-upload.s3.amazonaws]
3. **Graph & routing core**
   - Graph abstraction (NetworkX or igraph).
   - Baseline algorithms (single-objective Dijkstra variants).Carbon-Aware_Adaptive_Routing_Protocol_with_Real-Time_Traffic_Information_for_Cyber-Physical_Int.pdf+1
   - Multi-objective routers: Pareto labeling, weighted sum, maybe hybrid PSO–GA.Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1
   - Clean interfaces like `compute_routes(source, target, objectives, prefs)`.
4. **Scenario / traffic module (optional)**
   - Simple congestion multipliers by time of day or random factors, as in.[ppl-ai-file-upload.s3.amazonaws]
   - Could just be a function that maps base travel time to current time.
5. **API/UI layer**
   - Minimal web API (FastAPI/Flask) that takes: source, target, vehicle type, preference (eco vs fast), returns a small set of routes with metrics.
   - Simple web map (Leaflet/Folium/OSM tiles) or CLI for visualization.Intelligent_Dynamic_Routing_and_Optimization_with_Carbon_Awareness_for_Sustainable_and_E
6. **Experiment runner**
   - Scripts to load OD pairs, run all algorithms, log metrics to CSV, and generate plots.

This modularity also makes it easy to pitch as "framework" rather than just code.

---

4. Algorithms to implement and compare

Baseline algorithms

1. **Shortest distance path**
   - Dijkstra with weight = distance.Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1
2. **Fastest time path (no emissions)**
   - Dijkstra with weight = free-flow or congested travel time.[ppl-ai-file-upload.s3.amazonaws]
3. **Static carbon-aware path (scalar cost)**
   - Dijkstra with weight = simple emissions cost per edge (distance × constant factor or length×type factor), similar to static carbon-aware algorithm in and PSO-GA's objective.Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1
4. **Weighted sum multi-objective path**
   - Dijkstra with weight = wd·distance+wt·time+we·emissionsw_d \cdot \text{distance} + w_t \cdot \text{time} + w_e \cdot \text{emissions}wd·distance+wt·time+we·emissions.
   - This mirrors the weighted multi-objective route in and is a strong baseline.[ppl-ai-file-upload.s3.amazonaws]

Advanced / multi-objective methods

Pick 1–2 from below; more will explode your scope.

1. **Multi-objective label-setting / label-correcting (MO-Dijkstra / NAMOA-like)**
   - Maintains Pareto sets of labels at each node for (distance, time, emissions).[ppl-ai-file-upload.s3.amazonaws]
   - Returns a set of non-dominated routes for a given OD pair.
   - You can restrict max labels per node for tractability.
2. **Evolutionary / metaheuristic path optimizer (simplified vs MMOEA-CDP)**
   - Implement a *simpler* GA/PSO-GA that encodes paths as sequences of nodes and uses a multi-objective fitness function.Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1
   - You don't need full multimodal maintenance like MMOEA-CDP; your goal is to show trade-offs vs MO-Dijkstra.
3. **Carbon-aware adaptive routing with traffic (optional)**
   - A simplified version of "Carbon-Aware Adaptive Algorithm" from.[ppl-ai-file-upload.s3.amazonaws]
   - Recompute edge weights using normalized time and emissions, run Dijkstra, maybe under different "carbon_weight" parameters to trace a Pareto curve.[ppl-ai-file-upload.s3.amazonaws]

Given 2–3 months, the most realistic set is:

- Baselines: distance, time, static carbon, weighted-sum.
- Advanced: MO-Dijkstra + one simple GA / PSO-GA.

---

5. Recommended tech stack and tools

- **Language:** Python (fast to prototype, rich ecosystem).Sustainable_and_Optimized_Route_Selection_using_Machine_Learning_Algorithm.pdf+3
- **Graph / routing:**
  - NetworkX (easy and sufficient for your scale).Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1
  - Optional: OSMnx to download and build graphs from OpenStreetMap.[ppl-ai-file-upload.s3.amazonaws]
- **Data / emissions:**

- Use simple emission factors from literature or public tables (e.g., g/km per vehicle type) to parameterize your model as in and.Modeling_and_Analysis_of_CO2_Emissions_of_Vehicles_through_Numerical_Simulation.pdf+1
- For traffic, either synthetic multipliers as in, or static speeds per road class.[ppl-ai-file-upload.s3.amazonaws]

- **Backend / experiments:**
  - Python scripts for experiments, Pandas for logging and analysis, Matplotlib/Seaborn for plots.
- **UI / integration (optional but nice):**
  - Flask/FastAPI + Leaflet/Folium for map visualization.Intelligent_Dynamic_Routing_and_Optimization_with_Carbon_Awareness_for_Sustainable_and_E

---

6. Datasets

You don't need proprietary data; use:

- **Road networks:**
  - OSMnx + OpenStreetMap for one or two cities (e.g., Bengaluru, Rome, Manhattan) as in the PSO-GA eco-routing paper.[ppl-ai-file-upload.s3.amazonaws]
  - Or synthetic benchmark graphs (grids, random, or CEC path-planning benchmarks) like in the MMMSPP paper.[ppl-ai-file-upload.s3.amazonaws]
- **Emissions data:**
  - Basic per-km emission factors per vehicle type from any public source; plug into your model similarly to and.Sustainable_and_Optimized_Route_Selection_using_Machine_Learning_Algorithm.pdf+1
  - If you include speed, approximate U-shaped speed-efficiency curve based on discussion in.[ppl-ai-file-upload.s3.amazonaws]
- **Traffic data (if included):**
  - Synthetic: time-of-day multipliers like peak/off-peak/mixed from.[ppl-ai-file-upload.s3.amazonaws]
  - No need for live feeds; just static scenarios.

---

7. Step-by-step implementation plan

Week 1–2: Foundations

1. Read in detail: the CPI carbon-aware adaptive routing paper, MMMSPP multi-objective evolutionary paper, PSO-GA eco-routing, and at least one ML-based eco-routing system paper.Intelligent_Dynamic_Routing_and_Optimization_with_Carbon_Awareness_for_Sustainable_and_Efficient.pdf
2. Decide core scope:
   - City, vehicle types, objectives (distance, time, emissions).
   - Which algorithms: e.g., Dijkstra variants + MO-Dijkstra + GA.

Week 2–3: Data & graph

1. Use OSMnx to download and simplify a city road network into a directed graph.[ppl-ai-file-upload.s3.amazonaws]
2. Precompute edge attributes: distance, base speed/time, road type.
3. Implement basic emission model: `edge_emission(edge, vehicle_type, traffic_state)` based on distance and a simple factor.Sustainable_and_Optimized_Route_Selection_using_Machine_Learning_Algorithm.pdf+1

Week 3–4: Baseline routing

1. Implement:
   - Distance-only Dijkstra.
   - Time-only Dijkstra (base time).

- Static carbon-only Dijkstra.
- Weighted sum Dijkstra.Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1

2. Build simple CLI/plot to show routes and metrics for a few OD pairs.

Week 4–6: Multi-objective & advanced

1. Implement MO-Dijkstra (Pareto label-setting) for 3 objectives.[ppl-ai-file-upload.s3.amazonaws]
2. Implement a simple GA or PSO-GA for route search, following a simplified version of the PSO-GA eco-routing setup.[ppl-ai-file-upload.s3.amazonaws]
3. Ensure: no invalid paths, termination criteria, and reasonable runtime on your graph size.

Week 6–7: Evaluation setup

1. Choose a set of OD pairs (e.g., 20–100 random pairs with distance ranges).
2. For each algorithm and OD pair, log: distance, time, emissions, runtime, number of Pareto solutions.
3. Generate plots:
   - Emissions vs time scatter (Pareto fronts).[ppl-ai-file-upload.s3.amazonaws]
   - Runtime vs graph size / OD count.Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1

Week 7–8: UI + polish

1. Build a minimal REST API or simple GUI to select OD, preferences, and visualize 2–3 suggested routes with metrics.Intelligent_Dynamic_Routing_and_Optimization_with_Carbon_Awareness_for_Sustainable_and_Efficient.p
2. Add configuration for vehicle type and preference (sliders for weights).

Week 8–10: Analysis & paper writing

1. Analyze:
   - Trade-offs between algorithms (quality vs runtime).Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+2
   - Effect of user weights on chosen route.
   - Benefit of Pareto set vs single weighted route.
2. Write paper/report, focusing on methodology, experiments, and clear figures.

---

8. Evaluation methodology

Datasets for evaluation

- 1–2 city graphs (one medium, one larger).[ppl-ai-file-upload.s3.amazonaws]
- Optionally a small synthetic grid to validate algorithm correctness.

Metrics

For each OD, record:

- **Route-level metrics:**
  - Distance (km).
  - Travel time (min) under base or congested speeds.[ppl-ai-file-upload.s3.amazonaws]
  - CO_2 emissions (g or kg) from your model.Sustainable_and_Optimized_Route_Selection_using_Machine_Learning_Algorithm.pdf+1
- **Algorithmic metrics:**
  - Runtime (ms / s).Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1

- Memory (rough estimate) if possible.
- Number of Pareto solutions for multi-objective algorithms.[ppl-ai-file-upload.s3.amazonaws]

Comparative experiments

Example experiments:

1. **Single-solution vs Pareto set quality**
   - Compare best weighted-sum route vs Pareto routes: how much emissions can you save for a small time penalty?Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1
2. **Exact vs metaheuristic**
   - Compare MO-Dijkstra vs GA/PSO-GA on same OD set:
     - How close is GA to Pareto front (distance in objective space)?
     - Runtime differences.Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1
3. **Robustness scenarios (optional)**
   - Randomly "remove" edges to simulate disruptions, see whether having multiple stored routes improves success / re-routing overhead.[ppl-ai-file-upload.s3.amazonaws]
4. **Scalability**
   - Increase graph size (via OSM area) and OD count, measure runtime scaling.Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1

---

9. Possible research contributions (publishable)

With careful experiments, you can claim:

- A **modular, graph-based framework** for carbon-aware multi-objective routing that exposes both single and multi-route outputs for navigation use (unlike some works focused entirely on logistics or path planning competitions).Carbon-Aware_Adaptive_Routing_Protocol_with_Real-Time_Traffic_Information_for_Cyber-Physical_Int.pdf+2
- An **empirical comparison** of exact multi-objective label-setting vs hybrid/metaheuristic routing for eco-routing on real urban networks using OpenStreetMap.Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1
- A **user-oriented preference and Pareto-routing model** showing how small changes in user weights affect emissions–time-distance trade-offs in practice, with quantification.Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1
- A **simple but transparent emissions model** integrated into routing, demonstrating how speed-sensitive factors affect route selection across scenarios.Modeling_and_Analysis_of_CO2_Emissions_of_Vehicles_through_Numerical_Simulation.pdf+1

That's enough for a student conference or workshop paper.

---

10. Key risks and challenges

- **Algorithm complexity:** Multi-objective label-setting can blow up in state size. Limit graph size and objectives, and prune dominated labels aggressively.[ppl-ai-file-upload.s3.amazonaws]
- **Metaheuristic tuning:** GA/PSO parameters can take time to tune; keep that part simple and secondary.[ppl-ai-file-upload.s3.amazonaws]
- **Data realism vs simplicity:** Over-complex emission models or traffic models will slow you down. Use simple but justifiable models.Intelligent_Dynamic_Routing_and_Optimization_with_Carbon_Awareness_for_Sustainable_and_Efficient.p

- **Scope creep:** Adding ML, full real-time traffic, or multi-modal (public transport, walking, etc.) will quickly exceed 2–3 months.Sustainable_and_Optimized_Route_Selection_using_Machine_Learning_Algorithm.pdf+1

---

11. Example final paper title and outline

Example title

> "Graph-Based Multi-Objective Carbon-Aware Routing for Urban Navigation: Pareto-Optimal Path Computation and Empirical Evaluation on Real Road Networks"

Outline

1. **Introduction**
   - Motivation: transport emissions, limitations of shortest-path routing, need for multi-objective eco-routing.Intelligent_Dynamic_Routing_and_Optimization_with_Carbon_Awareness_for_Sustainable_and_Efficier
   - Contributions: framework, algorithms, evaluation.
2. **Related Work**
   - Carbon-aware adaptive routing and multi-objective CPI frameworks.[ppl-ai-file-upload.s3.amazonaws]
   - PSO-GA eco-routing on OSM networks.[ppl-ai-file-upload.s3.amazonaws]
   - Multi-objective multi-point shortest path and MMOEA methods.[ppl-ai-file-upload.s3.amazonaws]
   - ML-based eco journey trackers and dashboards.Sustainable_and_Optimized_Route_Selection_using_Machine_Learning_Algorithm.pdf+1
3. **Problem Formulation**
   - Graph model, objectives (distance, time, emissions), emission model.Modeling_and_Analysis_of_CO2_Emissions_of_Vehicles_through_Numerical_Simulation.pdf+1
   - Definition of Pareto optimal routes and weighted-sum routes.[ppl-ai-file-upload.s3.amazonaws]
4. **Methods**
   - Baseline single-objective and weighted-sum Dijkstra.Carbon-Aware_Adaptive_Routing_Protocol_with_Real-Time_Traffic_Information_for_Cyber-Physical_Int.pdf+1
   - Multi-objective label-setting algorithm.[ppl-ai-file-upload.s3.amazonaws]
   - Optional GA/PSO-GA path optimizer.[ppl-ai-file-upload.s3.amazonaws]
   - System architecture and modular design.Intelligent_Dynamic_Routing_and_Optimization_with_Carbon_Awareness_for_Sustainable_and_Efficien
5. **Experimental Setup**
   - Datasets (cities, graph stats, OD sets).Eco-Routing_in_Urban_Networks_for_Smart_Cities_A_Hybrid_PSO-GA_Approach_for_Low-Carbon_Traffic_P.pdf+1
   - Vehicle types, emission factors, traffic scenarios.Sustainable_and_Optimized_Route_Selection_using_Machine_Learning_Algorithm.pdf+1
   - Metrics and implementation details.
6. **Results**
   - Emissions vs time/distance trade-off plots.Carbon-Aware_Adaptive_Routing_Protocol_with_Real-Time_Traffic_Information_for_Cyber-Physical_Int.pdf+1
   - Runtime and scalability comparison.Multi-Objective_Optimization_for_Multimodal_Multi-Objective_Multi-Point_Shortest_Path_Problem_Co.pdf+1
   - Impact of user preferences and multiple routes.Sustainable_and_Optimized_Route_Selection_using_Machine_Learning_Algorithm.pdf+1
7. **Discussion**
   - Practical implications for navigation systems.
   - Limitations and potential extensions (real-time data, multimodal, ML).

8. **Conclusion and Future Work**
    - Summary of findings, suggestions for next steps.Intelligent_Dynamic_Routing_and_Optimization_with_Carbon_Awareness_for_Sustainable_and_Efficient.

If you had to pick *one* main novelty axis, would you prefer to focus on (a) Pareto route computation and visualization, (b) comparing exact vs metaheuristic eco-routing, or (c) a more applied system with a strong dashboard and user preference modeling?

▼ Tools

1️⃣ Data & Map Sources

2️⃣ Graph Processing Tools

3️⃣ Algorithms & Optimization Techniques

4️⃣ Simulation & Experiment Tools

5️⃣ Visualization & Evaluation

I'll list **the essential ones first**, then optional advanced ones.

---

1. Programming & Core Environment

These are the **foundation tools**.

Programming Language

- **Python** ⭐ (strongly recommended)

Why:

- excellent graph libraries
- widely used in transportation research
- easy integration with datasets and maps

Python Libraries

Essential ones:

- **NetworkX** → graph algorithms
- **OSMnx** → download road networks
- **NumPy** → numerical operations
- **Pandas** → data analysis

Example stack:

```
Python
│
├── NetworkX
├── OSMnx
├── NumPy
└── Pandas
```

---

2. Map & Transportation Data Tools

Your project needs **real road network data**.

OpenStreetMap (OSM)

Free global map dataset.

Provides:

- road networks

- speed limits
- road types
- intersections

OSMnx

Python library that converts OpenStreetMap into a **graph**.

Example concept:

```
OpenStreetMap data
        ↓
OSMnx
        ↓
Graph:
Nodes = intersections
Edges = roads
```

---

3. Graph Modeling Tools

Your routing system depends on **graph theory**.

NetworkX

Main graph library.

Used for:

- shortest path
- graph traversal
- network analysis

Example operations:

- shortest path
- edge weighting
- graph metrics

Graph Representation

Your system models roads as:

```
Graph G = (V, E)

V = intersections
E = roads
```

Routing problem:

$$\min_{P}\sum_{e \in P} w_e$$

Where:

- (P) = path
- (w_e) = edge weight

In your case:

```
w = distance + emissions + traffic penalty
```

---

4. Algorithms You Need

These are the **core techniques**.

Exact Algorithms

These guarantee optimal routes.

Essential ones:

- **Dijkstra Algorithm**
- **A∗ Algorithm**
- **Multi-objective shortest path**

Use them as **baseline algorithms**.

Multi-Objective Optimization

Because your route must balance:

- distance
- travel time
- emissions

Common techniques:

- weighted sum method
- Pareto optimal routing

Metaheuristic Algorithms (optional but good for research)

Used for large networks.

Examples:

- **Genetic Algorithm**
- **Ant Colony Optimization**
- **Particle Swarm Optimization**

These help compare:

```
Exact algorithms vs heuristic methods
```

---

5. Carbon Emission Modeling

You must estimate **CO_2 emissions per road segment**.

Simple models use:

```
emissions = emission_rate × distance
```

Better models consider:

- vehicle speed
- traffic congestion
- acceleration
- road type

Datasets can come from:

- vehicle emission studies
- transportation research datasets

---

6. Simulation & Experiment Tools

You need to run experiments.

Example experiment:

```
simulate 5000 trips in a city
compare algorithms
```

Metrics to evaluate:

- total emissions
- travel time
- runtime
- scalability

Tools:

- Python scripts
- Jupyter notebooks

---

7. Visualization Tools

These make your results understandable.

Map Visualization

Tools:

- **Folium** (Python map library)
- **Leaflet.js**
- **Matplotlib**

Example output:

- shortest route
- eco-friendly route

Displayed on a city map.

---

8. Data Analysis Tools

Used for research experiments.

Tools:

- **Pandas**
- **NumPy**
- **Matplotlib**
- **Seaborn**

Used for graphs like:

- emission comparison
- runtime comparison

---

9. Optional Advanced Tools

If you want a stronger system.

Traffic Data

Sources:

- city traffic datasets
- Google traffic APIs

Adds realism.

Routing Engines

Industrial tools:

- GraphHopper
- Valhalla
- OpenRouteService

These are heavy systems used in real navigation apps.

Not necessary for a student project.

---

10. Complete Tool Stack (Recommended)

Simple and effective stack:

```
Python
│
├── OSMnx (road network data)
├── NetworkX (graph algorithms)
├── NumPy / Pandas (data processing)
├── Matplotlib / Seaborn (analysis)
└── Folium (map visualization)
```

---

11. Core Techniques Used in the Project

From a research perspective, you will use:

Discrete Mathematics

- graph theory
- weighted graphs
- path optimization

Algorithms

- shortest path algorithms
- multi-objective optimization
- heuristic search

Network Modeling

- transportation networks
- routing systems

Key Insight

Your project is essentially:

```
Road Network Graph
        +
```

```
Routing Algorithms
         +
Emission Cost Model
         =
Carbon-Aware Navigation System
```

If you want, I can also show you something **very useful for starting the project quickly**:

I can give you the **exact step-by-step implementation roadmap (like a research lab would plan it)** so you know exactly what to build first.